



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский государственный исследовательский университет»

ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК И ТЕХНОЛОГИЙ

НАПРАВЛЕНИЕ ПОДГОТОВКИ 01.03.02 Прикладная математика и информатика

О Т Ч Е Т

По лабораторной работе № 4

Название: Введение в CUDA. Операции с массивами.
Сортировка слиянием

Дисциплина: Параллельные вычислительные системы

Студент

ИТ-3,4

(Группа)

(Подпись, дата)

Н.М. Коньшин

(И.О. Фамилия)

Преподаватель

А.С. Белозеров

(Подпись, дата)

(И.О. Фамилия)

Пермь, 2025

Введение

Цель работы: приобретение знаний, умений и навыков в области технологии параллельного программирования с использованием графических процессоров в качестве ускорителей вычислений на примере языка CUDA C.

Задание работы

1. Изучить основы технологии параллельного программирования с использованием графических процессоров в качестве ускорителей вычислений на примере языка CUDA C.
2. Используя средства ГПУ, научиться решать задачи сложения (и других арифметических операций) элементов.
3. Используя средства ГПУ, научиться реализовывать параллельную работу сортировки элементов слиянием.

Основная часть

- Задания данной лабораторной работы частично пересекаются с заданиями ЛР №2 и ЛР №3. Поэтому часть скриптов и написанных программ была взята именно оттуда. Весь дальнейший код программ представлен на [Github в моем репозитории](#).

Таблица сравнения результатов работы ЛР №4

	Количество элементов	Время выполнения (сек) Macbook Pro Последовательная версия	Время выполнения (сек) Kepler		Самая быстрая версия по итогам тестов
			Последовательная версия	Параллельная версия (CUDA)	
Task 1	5.000.000	0.4803	0.7778	0.9356	Macbook Pro
Task 2	1.000.000	0.0542	0.1710	0.0114	Kepler (Cuda)
	5.000.000	0.2973	0.9450	0.0905	Kepler (Cuda)
	10.000.000	0.6497	1.9770	0.1929	Kepler (Cuda)
Task 3	5.000.000	1.0034	1.7307	0.0019	Kepler (Cuda)
Task 4	4.000.000	0.4940	1.1935	1.1130	Macbook Pro

Task 1

- Результаты выполнения программ локально и на «ПГНИУ-Кеплер»

```
~/Documents/PSU/2.4sem/Параллельные вычислительные системы/Labs_git/LR4/Task1  main ./sequential_sum
Размер массива: 5000000 элементов
Сумма элементов: 2502235841
Время выполнения: 0.480288 секунд
```

```
array_generation.py array.txt cuda_sum
[nkonshin.me.com@node5 Task1]$ ./cuda_sum
Размер массива: 5000000 элементов
Сумма элементов: 2502235841
Время выполнения: 0.935645 секунд
Конфигурация: 19532 блоков по 256 потоков
[nkonshin.me.com@node5 Task1]$
```

- Самой быстрой оказалась последовательная версия на MacBook Pro (0,48 с против 0,78 с — Kepler CPU и 0,94 с — Kepler CUDA). Для одной редукции накладные расходы на копирование данных и запуск ядра на старом GPU Kepler перекрывают выгоду от параллелизма, а высокое IPC и широкие векторные блоки M1 Pro позволяют быстро обработать массив на CPU.

Task 2

- Результаты выполнения программ локально и на «ПГНИУ-Кеплер»

```
~/Documents/PSU/2.4sem/Параллельные вычислительные системы/Labs_git/LR4/Task2  main ± ./sequential_merge_sort array_1mln.txt
Сортировка массива из 1000000 элементов...
Массив успешно отсортирован за 0.0542 секунд

Первые 10 элементов:
1 1 1 1 1 1 1 1 1 1

Последние 10 элементов:
1000 1000 1000 1000 1000 1000 1000 1000 1000 1000
~/Documents/PSU/2.4sem/Параллельные вычислительные системы/Labs_git/LR4/Task2  main ± ./sequential_merge_sort array_5mln.txt
Сортировка массива из 5000000 элементов...
Массив успешно отсортирован за 0.2973 секунд

Первые 10 элементов:
1 1 1 1 1 1 1 1 1 1

Последние 10 элементов:
1000 1000 1000 1000 1000 1000 1000 1000 1000 1000
~/Documents/PSU/2.4sem/Параллельные вычислительные системы/Labs_git/LR4/Task2  main ± ./sequential_merge_sort array_10mln.txt
Сортировка массива из 10000000 элементов...
Массив успешно отсортирован за 0.6497 секунд

Первые 10 элементов:
1 1 1 1 1 1 1 1 1 1

Последние 10 элементов:
1000 1000 1000 1000 1000 1000 1000 1000 1000 1000
~/Documents/PSU/2.4sem/Параллельные вычислительные системы/Labs_git/LR4/Task2  main ±
```

```
[nkonshin.me.com@node5 Task2]$ ./bitonic_sort array_1mln.txt
Массив успешно отсортирован за 0.0114 секунд
[nkonshin.me.com@node5 Task2]$ ./bitonic_sort array_5mln.txt
Массив успешно отсортирован за 0.0905 секунд
[nkonshin.me.com@node5 Task2]$ ./bitonic_sort array_10mln.txt
Массив успешно отсортирован за 0.1929 секунд
[nkonshin.me.com@node5 Task2]$
```

```

[nkonshin.me.com@node5 Task2]$ ./sequential_merge_sort array_1mln.txt
Сортировка массива из 1000000 элементов...
Массив успешно отсортирован за 0.1710 секунд

Первые 10 элементов:
1 1 1 1 1 1 1 1 1 1

Последние 10 элементов:
1000 1000 1000 1000 1000 1000 1000 1000 1000 1000
[nkonshin.me.com@node5 Task2]$ ./sequential_merge_sort array_5mln.txt
Сортировка массива из 5000000 элементов...
Массив успешно отсортирован за 0.9450 секунд

Первые 10 элементов:
1 1 1 1 1 1 1 1 1 1

Последние 10 элементов:
1000 1000 1000 1000 1000 1000 1000 1000 1000 1000
[nkonshin.me.com@node5 Task2]$ ./sequential_merge_sort array_10mln.txt
Сортировка массива из 10000000 элементов...
Массив успешно отсортирован за 1.9770 секунд

Первые 10 элементов:
1 1 1 1 1 1 1 1 1 1

Последние 10 элементов:
1000 1000 1000 1000 1000 1000 1000 1000 1000 1000
[nkonshin.me.com@node5 Task2]$ █

```

- Во всех трех вариантах лидирует Bitonic Sort на Kepler CUDA (ускорение 3–10х относительно CPU). Битоническая сортировка полностью параллельна, все потоки заняты вычислениями, а пропускная способность глобальной памяти GPU выше, чем у CPU. Рост объёма данных лишь повышает эффективность GPU, потому что время копирования растёт линейно, а вычислительное ускорение — близко к квадратичному числу потоков.

Task 3

- Результаты выполнения программ локально и на «ПГНИУ-Кеплер»

```
~/Documents/PSU/2.4sem/Параллельные вычислительные системы/Labs_git/LR4/Task3 > ./sequential_array_ops
=== ПОСЛЕДОВАТЕЛЬНАЯ ВЕРСИЯ ===
Размер массивов: 5000000 элементов
Время выполнения: 1.003425 секунд

Сумма (первые 20 из 5000000):
1753.00 | 518.00 | 669.00 | 1299.00 | 843.00
1207.00 | 940.00 | 1546.00 | 1411.00 | 562.00
1262.00 | 457.00 | 58.00 | 1071.00 | 255.00
822.00 | 1524.00 | 1114.00 | 1517.00 | 1285.00

Разность (первые 20 из 5000000):
-39.00 | -348.00 | -29.00 | 45.00 | 503.00
613.00 | 24.00 | 390.00 | -505.00 | -134.00
-658.00 | 177.00 | -38.00 | -543.00 | -199.00
470.00 | 220.00 | 226.00 | -87.00 | 375.00

Произведение (первые 20 из 5000000):
767872.00 | 36805.00 | 111680.00 | 421344.00 | 114410.00
270270.00 | 220756.00 | 559504.00 | 433974.00 | 74472.00
289920.00 | 44380.00 | 480.00 | 213048.00 | 6356.00
113696.00 | 568544.00 | 297480.00 | 573430.00 | 377650.00

Частное (первые 20 из 5000000):
0.96 | 0.20 | 0.92 | 1.07 | 3.96
3.06 | 1.05 | 1.67 | 0.47 | 0.61
0.31 | 2.26 | 0.21 | 0.33 | 0.12
3.67 | 1.34 | 1.51 | 0.89 | 1.82
```

```
[nkonshin.me.com@node5 Task3]$ ./sequential_array_ops
=== ПОСЛЕДОВАТЕЛЬНАЯ ВЕРСИЯ ===
Размер массивов: 5000000 элементов
Время выполнения: 1.730650 секунд

Сумма (первые 20 из 5000000):
1753.00 | 518.00 | 669.00 | 1299.00 | 843.00
1207.00 | 940.00 | 1546.00 | 1411.00 | 562.00
1262.00 | 457.00 | 58.00 | 1071.00 | 255.00
822.00 | 1524.00 | 1114.00 | 1517.00 | 1285.00

Разность (первые 20 из 5000000):
-39.00 | -348.00 | -29.00 | 45.00 | 503.00
613.00 | 24.00 | 390.00 | -505.00 | -134.00
-658.00 | 177.00 | -38.00 | -543.00 | -199.00
470.00 | 220.00 | 226.00 | -87.00 | 375.00

Произведение (первые 20 из 5000000):
767872.00 | 36805.00 | 111680.00 | 421344.00 | 114410.00
270270.00 | 220756.00 | 559504.00 | 433974.00 | 74472.00
289920.00 | 44380.00 | 480.00 | 213048.00 | 6356.00
113696.00 | 568544.00 | 297480.00 | 573430.00 | 377650.00

Частное (первые 20 из 5000000):
0.96 | 0.20 | 0.92 | 1.07 | 3.96
3.06 | 1.05 | 1.67 | 0.47 | 0.61
0.31 | 2.26 | 0.21 | 0.33 | 0.12
3.67 | 1.34 | 1.51 | 0.89 | 1.82
```

```

[nkonshin.me.com@node5 Task3]$ ./cuda_array_ops
=== ПАРАЛЛЕЛЬНАЯ ВЕРСИЯ (CUDA) ===
Размер массивов: 5000000 элементов
Время выполнения: 0.001859 секунд

Сумма (первые 20 из 5000000):
1753.00 | 518.00 | 669.00 | 1299.00 | 843.00
1207.00 | 940.00 | 1546.00 | 1411.00 | 562.00
1262.00 | 457.00 | 58.00 | 1071.00 | 255.00
822.00 | 1524.00 | 1114.00 | 1517.00 | 1285.00

... и еще 4999980 элементов

Разность (первые 20 из 5000000):
-39.00 | -348.00 | -29.00 | 45.00 | 503.00
613.00 | 24.00 | 390.00 | -505.00 | -134.00
-658.00 | 177.00 | -38.00 | -543.00 | -199.00
470.00 | 220.00 | 226.00 | -87.00 | 375.00

... и еще 4999980 элементов

Произведение (первые 20 из 5000000):
767872.00 | 36805.00 | 111680.00 | 421344.00 | 114410.00
270270.00 | 220756.00 | 559504.00 | 433974.00 | 74472.00
289920.00 | 44380.00 | 480.00 | 213048.00 | 6356.00
113696.00 | 568544.00 | 297480.00 | 573430.00 | 377650.00

... и еще 4999980 элементов

Частное (первые 20 из 5000000):
0.96 | 0.20 | 0.92 | 1.07 | 3.96
3.06 | 1.05 | 1.67 | 0.47 | 0.61
0.31 | 2.26 | 0.21 | 0.33 | 0.12
3.67 | 1.34 | 1.51 | 0.89 | 1.82

... и еще 4999980 элементов

```

- GPU даёт колоссальное ускорение (0,002 с на Kepler CUDA против 1,0 с — M1 и 1,73 с — Kepler CPU). Каждая операция ($\pm$, $\times$, $\div$) независима, поэтому тысячи потоков на GPU выполняют их параллельно, а объём передаваемых данных относительно мал. За счёт этого все миллионы действий выполняются одновременно, и выгода от параллелизма огромна.

Task 4

- Результаты выполнения программ локально и на «ПГНИУ-Кеплер»

```

~/Documents/PSU/2.4sem/Параллельные вычислительные системы/Labs_git/LR4/Task4 main ± ./sequential_matrix_ops
=== ПОСЛЕДОВАТЕЛЬНАЯ ВЕРСИЯ ===
Размер матриц: 2000x2000 (всего 4000000 элементов)
Время выполнения: 0.493970 секунд
Скорость: 32390631.01 операций/сек
Первые 5 результатов операций:

```

Индекс	Сложение	Вычитание	Умножение	Деление
1	702.00	568.00	42545.00	9.48
2	684.00	-638.00	15203.00	0.03
3	863.00	-653.00	79590.00	0.14
4	286.00	-180.00	12349.00	0.23
5	1313.00	657.00	323080.00	3.00

```

Всего обработано элементов: 4000000

```

```
[nkonshin.me.com@node5 Task4]$ ./sequential_matrix_ops
=== ПОСЛЕДОВАТЕЛЬНАЯ ВЕРСИЯ ===
Размер матриц: 2000x2000 (всего 4000000 элементов)
Время выполнения: 1.193499 секунд
Скорость: 13405960.12 операций/сек
Первые 5 результатов операций:
```

Индекс	Сложение	Вычитание	Умножение	Деление
1	702.00	568.00	42545.00	9.48
2	684.00	-638.00	15203.00	0.03
3	863.00	-653.00	79590.00	0.14
4	286.00	-180.00	12349.00	0.23
5	1313.00	657.00	323080.00	3.00

```
Всего обработано элементов: 4000000
```

```
[nkonshin.me.com@node5 Task4]$ ./cuda_matrix_ops
=== ПАРАЛЛЕЛЬНАЯ ВЕРСИЯ (CUDA) ===
Размер матриц: 2000x2000 (всего 4000000 элементов)
Время выполнения: 1.114249 секунд
Скорость: 14359442.99 операций/сек
Первые 5 результатов операций:
```

Индекс	Сложение	Вычитание	Умножение	Дение
1	702.00	568.00	42545.00	9.48
2	684.00	-638.00	15203.00	0.03
3	863.00	-653.00	79590.00	0.14
4	286.00	-180.00	12349.00	0.23
5	1313.00	657.00	323080.00	3.00

```
Всего обработано элементов: 4000000
```

- Здесь вновь выигрывает CPU M1 Pro (0,49 с против 1,11 с на Kepler CUDA). Нужно копировать две входные и четыре выходные матрицы, а также запускать несколько ядер подряд, что увеличивает накладные расходы на GPU. У M1 Pro высокая пропускная способность системной памяти (Unified Memory), и в итоге процессор, работающий с ней напрямую заканчивает работу быстрее.

Вывод

В ходе лабораторной работы № 4 были реализованы и исследованы четыре пары программ — последовательных и параллельных. Задачи охватывали типичные сценарии массовой обработки данных: вычисление суммы большого одномерного массива, сортировка, покомпонентные операции над двумя массивами и аналогичные операции для двумерных матриц.

Эксперименты подтвердили, что выбор оптимальной архитектуры зависит прежде всего от характера задачи и объёма передаваемых данных. Для вычислительно «лёгкой» редукции (Task 1) и матричных операций с относительно небольшим объёмом данных (Task 4) наилучшее время показал CPU M1 Pro: выигрыш обеспечили высокая однопоточная производительность и отсутствие затрат на копирование памяти между хостом и устройством. Напротив, при сортировке массивов размером 5–10 млн элементов (Task 2) и при параллельной обработке двух одномерных массивов (Task 3) значительное ускорение — до десятикратного — достигнуто на GPU Kepler. В обоих случаях работа легко декомпозируется на множество независимых операций, что позволяет загрузить тысячи потоков графического процессора и компенсировать накладные расходы на передачу данных.

Таким образом, GPU-ускорение целесообразно применять, когда:

- объём данных превышает сотни тысяч элементов;
- над каждым элементом выполняется простая независимая операция или шаг алгоритма;
- число обращений к глобальной памяти невелико по сравнению с количеством вычислений.

В остальных случаях современный CPU с широкой SIMD-архитектурой и высокой частотой способен обеспечить сопоставимую или даже лучшую производительность при меньшей сложности кода.

Теоретическая справка

1. CPU vs GPU

- CPU (Central Processing Unit) рассчитан на универсальные задачи, имеет несколько мощных ядер, сложную иерархию кешей и высокую тактовую частоту.
- GPU (Graphics Processing Unit) содержит сотни-тысячи лёгких ядер (ALU) и высокую пропускную способность памяти. Он идеален для задач, в которых одна и та же операция применяется к большому объёму данных

2. Архитектура NVIDIA CUDA

CUDA (Compute Unified Device Architecture) — программная платформа NVIDIA для написания общевычислительных программ под GPU на языке, близком к C. Ключевые понятия:

- Поток (thread) — наименьшая единица выполнения.
- Блок (block) — группа потоков, имеющих общий fast-shared-memory. Число потоков в блоке обычно кратно 32 (warp).
- Сетка (grid) — набор блоков, порождённых одним вызовом ядра (kernel).

Хост-код (CPU) выделяет память, копирует данные на устройство, запускает ядро и копирует результаты обратно. Каждый вызов указывает размер сетки и блока.

4. Типичные шаблоны параллельных алгоритмов

4.1. Редукция

Свёртка множества элементов к одному значению (сумма, максимум).

Алгоритм строится как ступенчатое «дерево» сложений внутри блока (shared memory) и, при необходимости, между блоками.

4.2. Параллельная сортировка Bitonic

Bitonic Sort строится из серии сравнений-перестановок, которые можно выполнять параллельно, так как порядок операций не зависит от данных. Сложность $O(n \log^2 n)$, но практически эффективна для GPU благодаря регулярному шаблону доступа.

4.3. Покоэлементные операции

Сложение, вычитание, умножение или деление элементов двух массивов (или матриц) с одинаковыми индексами естественным образом

распараллеливаются: каждому элементу соответствует отдельный поток.

Задача идеально «масштабируется» на тысячи ядер GPU.

OpenMP vs MPI vs CUDA

1. Модель параллелизма

- OpenMP — директивы «расщепляют» цикл или участок кода на потоки внутри одного узла (многопроцессор или многоядерный CPU). Память общая, потоки видят одно адресное пространство.
- MPI — запуск отдельного процесса на каждом узле кластера; процессы обмениваются сообщениями (send/recv). Память распределённая, поэтому программист явно описывает передачу данных.
- CUDA — запускает тысячи лёгких потоков на GPU; память хоста и память устройства разделены, копирование данных требует явных вызовов.

2. Простота разработки

OpenMP считается «самым простым стартом»: часто достаточно добавить пару директив `#pragma omp parallel for`. Код остаётся почти неизменным.

MPI требует продуманного обмена данными, коллективных операций, учёта топологии сети — входной порог выше. CUDA предполагает изучение архитектуры GPU (сетки, блоки, потоки, виды памяти) и careful-оптимизаций, но предоставляет высокоуровневый C-подобный синтаксис.

3. Масштабируемость

OpenMP ограничен одним узлом (обычно до 64–128 ядер). Дальнейший рост упирается в пропускную способность общей памяти.

MPI масштабируется на тысячи узлов, пока сеть выдерживает объём сообщений.

CUDA масштабируется внутри одного GPU (до ~100к ядер) и, при

использовании Multi-GPU, до нескольких устройств, но внутри одного узла.

4. Класс задач

OpenMP подходит для лёгкой распараллельной обработки циклов, численных методов с плотным обменом между потоками.

MPI незаменим для задач, требующих огромной совокупной памяти или естественно разбиваемых на подзадачи (метод конечных элементов, распределённые базы данных).

CUDA эффективна в задачах с большим числом одинаковых, независимых операций над массивами/матрицами: линейная алгебра, обработка изображений, машинное обучение.

5. Производительность

На малом количестве ядер OpenMP даёт «дешёвое» ускорение без накладных расходов сети.

MPI выигрывает за счёт суммарной мощности кластера, но затрачивает время на передачу сообщений; эффективность зависит от объёма данных и латентности сети.

CUDA обеспечивает другой порядок ускорения на «массово-параллельных» алгоритмах, но чувствительна к времени копирования данных и размерам задачи.

6. Переносимость

OpenMP и MPI работают на большинстве CPU-архитектур; CUDA привязана к GPU NVIDIA.

7. Итоговый выбор

- Нужен быстрый прирост внутри одного сервера — берём OpenMP.
- Задача расходится на независимые блоки и требует много памяти — выбираем MPI-кластер.
- Есть вычислительно плотные, однотипные операции и доступ к GPU — используем CUDA.